IESPORTS

Tu instituto, tu torneo, tu victoria

10 JUNIO 2025 2ºDAW

SILVIA APARICIO MARTÍN,
JUAN SEBASTIÁN OSORIO MONTAÑO Y
ERIK DANIEL BERMEJO BJERVED

CC BY NC

ÍNDICE

0. INTRODUCCIÓN....4
1. NATURALEZA DEL PROYECTO....4
1.1 DESCRIPCIÓN DEL PROYECTO....4
1.2 JUSTIFICACIÓN/FUNDAMENTACIÓN....5
1.3 BENEFICIARIOS DEL PROYECTO....5
2. DEFINICIÓN DE OBJETIVOS....6
2.1 OBJETIVO PRINCIPAL....6
2.2.1 DISEÑO Y ARQUITECTURA DEL SISTEMA....6
2.2.2 IMPLEMENTACIÓN DEL BACK-END....7
2.2.3 IMPLEMENTACIÓN DEL FRONT-END....7
2.2.4 INTERFAZ DE USUARIO....8
2.2.5 DESPLIEGUE Y DOCUMENTACIÓN....8
2.2.6 MEJORA DE LA COMUNICACIÓN Y SEGUIMIENTO....8
3. RECURSOS....9
4. TAREAS....10
5. DIAGRAMA TEMPORAL....11
6. CAPÍTULOS DE DESARROLLO....12
6.1 DIAGRAMA DE CASOS DE USO....12
6.2 DIAGRAMA DE CLASES....12
6.3 DIAGRAMA RELACIONAL....13
6.4 DIAGRAMA E/R....14
6.5 INNOVACIONES....15
6.5.1 INNOVACIONES FRONTEND....15
6.5.2 INNOVACIONES BACKEND....20
6.6 CÓDIGO Y ESTRUCTURA....27
6.6.1 ESTRUCTURA FRONTEND....27
7. RESULTADOS OBTENIDOS....33
7.1 PROBLEMAS Y SUS SOLUCIONES....33

8. CONCLUSIONES Y PROPUESTAS..... 34
8.1 POSIBLES MEJORAS..... 34
8.2 POSIBLES AMPLIACIONES..... 35
9- BIBLIOGRAFÍA Y REFERENCIAS..... 35
10- ANEXOS..... 36
10.1 PROTOTIPO DE IESPORTS..... 36
10.3 LOGO..... 39
10.3.1 LOGO..... 39
..... 39
10.3.2 ESLOGAN..... 39
10.2.3 LOGO CON ESLOGAN..... 39

0. INTRODUCCIÓN

- Nuestro Trabajo de Fin de Grado es la ocasión perfecta para poner en práctica todos los conocimientos que hemos adquirido en este grado, desarrollando un proyecto que nos interese y nos motive, a la vez que reforzamos habilidades de organización, compromiso, esfuerzo y, sobre todo, trabajo en equipo.

- En este caso hemos decidido desarrollar **IESPORTS**, una aplicación web diseñada para que el administrador del centro educativo pueda crear y gestionar torneos, equipos y partidos de manera centralizada. Los alumnos se registran únicamente para acceder al contenido de la plataforma y, si desean formar parte de un equipo, contactan con el administrador, quien los añade a equipos, partidos y torneos. De esta forma, se garantiza un control adecuado de los participantes y se facilita el seguimiento dinámico de cada competición.

1. NATURALEZA DEL PROYECTO

1.1 DESCRIPCIÓN DEL PROYECTO

IESPORTS surge ante la falta de una herramienta única para que el centro educativo organice de forma ágil y ordenada sus competiciones: actualmente no existe un sistema centralizado para crear torneos, inscribir participantes ni consultar calendarios.

Con esta plataforma web, el administrador podrá definir torneos (título, fechas, etc), crear equipos y programar partidos en un calendario.

A su vez, los alumnos se registrarán para acceder como espectadores pudiendo visualizar fechas de partidos y de torneos con sus resultados sin intervención manual de listas impresas ni correos electrónicos. Cuando el administrador los incorporé, podrán participar en equipos, partidos y torneos y resultados de cada encuentro.

De esta manera, **IESPORTS** busca simplificar la gestión completa de competiciones, asegurar que la información esté siempre actualizada y facilitar el seguimiento en tiempo real tanto a organizadores como a estudiantes.

Para desarrollar esta aplicación web hemos usado para el frontend: Visual Studio con el framework Vue.js y el lenguaje TypeScript, y para el backend: SpringBoot con el lenguaje Java.

1.2 JUSTIFICACIÓN/FUNDAMENTACIÓN

Cuando evaluamos proyectos para el TFG, identificamos una necesidad real en el instituto: no existe una herramienta única para organizar competiciones, lo que obliga a usar papel, hojas de cálculo dispersas o correos electrónicos, generando errores, retrasos y falta de visibilidad para alumnos y profesores. Partiendo de esta carencia, decidimos crear **IESPORTS** para aportar valor inmediato al centro y garantizar que el sistema pueda mantenerse y escalarse en el futuro.

La motivación surge de nuestra experiencia: como estudiantes, hemos visto cómo la ausencia de un sistema centralizado retrasa la programación de partidos, complica la asignación de equipos y provoca confusión en los calendarios. Aunque existen plataformas genéricas de gestión deportiva, suelen ser complejas, orientadas al ámbito profesional y poco accesibles para un instituto. Por eso buscamos una solución sencilla, intuitiva y adaptada a los recursos técnicos del centro.

IESPORTS justifica su desarrollo al automatizar la creación de torneos, equipos y partidos, y al mejorar la comunicación entre organizadores y alumnos. Al centralizar toda la información en una base de datos y ofrecerla desde una interfaz web, se reduce el margen de error, se evita duplicidad de datos y los alumnos pueden consultar en tiempo real los calendarios oficiales.

Además, este proyecto nos permite aplicar en un caso práctico todos los conocimientos adquiridos en programación web o investigados (front-end con Vue e Ionic, back-end con Spring Boot, seguridad y bases de datos relacionales), reforzando nuestras competencias de análisis, diseño y trabajo en equipo.

1.3 BENEFICIARIOS DEL PROYECTO

- Administrador del centro educativo: podrá crear y gestionar torneos, equipos y partidos de forma centralizada, sin depender de papel ni hojas de cálculo o avisos por correos.
- Alumnos (espectadores): acceden a la plataforma para consultar calendario y resultados de los torneos cuando el administrador los añade, evitando confusiones y desinformación.
- Equipo directivo del centro: cuenta con una visión global de la actividad deportiva del instituto, pudiendo extraer informes y evaluar la participación de alumnos.

- Padres y tutores (indirectamente): al tener acceso a información actualizada (fecha y hora de partidos), pueden apoyar a los alumnos y asistir a eventos si lo desean.

1.4 LOCALIZACIÓN DEL PROYECTO: DÓNDE SE LLEVARÁ A CABO

IESPORTS se implementará en el propio centro educativo (por ejemplo, el *IES "Tetúan De Las Victorias"*), alojado en servidores locales o en la nube accesibles desde la red del instituto. Su cobertura alcanza a toda la comunidad escolar:

- **Alumnos**: todos los matriculados en el centro podrán registrarse como espectadores.
- **Administradores**: personal docente y de secretaría tendrán acceso completo para gestionar torneos.
- **Territorio**: ámbito local dentro del instituto, aunque al estar en la nube también pueden acceder desde casa o dispositivos móviles si se requiere.
- **Población afectada**: aproximadamente X alumnos e Y administradores (equivalente a toda la comunidad educativa), quienes podrán ver calendarios y resultados en tiempo real sin limitaciones físicas.

2. DEFINICIÓN DE OBJETIVOS

2.1 OBJETIVO PRINCIPAL

Nuestro objetivo es desarrollar una aplicación web que permita al administrador del centro educativo crear y gestionar torneos, equipos y partidos de forma práctica, intuitiva de usar y que ayude a hacer llegar la información tanto a los participantes de partidos como a los espectadores.

2.2 OBJETIVOS ESPECÍFICOS

2.2.1 DISEÑO Y ARQUITECTURA DEL SISTEMA

- Definir la estructura de la base de datos (MySQL) con tablas para usuarios, torneos, equipos, roles, deportes, cursos y partidos.
- Diseñar la API REST en Spring Boot que exponga endpoints para administrar las tablas anteriormente mencionadas.

2.2.2 IMPLEMENTACIÓN DEL BACK-END

Para la implementación del backend se ha utilizado Spring Boot, un framework moderno y eficiente que facilita la creación de aplicaciones empresariales en Java. El objetivo del backend es ofrecer una API REST que permita gestionar toda la lógica del sistema, desde usuarios y autenticación, hasta torneos, roles, deportes, equipos, cursos y partidos.

Además, se han utilizado diversas dependencias para facilitar funcionalidades clave como la persistencia con JPA, la validación de datos, la seguridad, el envío de correos, y la generación automática o manual de documentación con Swagger.

El backend expone los endpoints necesarios para el funcionamiento completo del sistema, permitiendo operaciones como:

- Registro e inicio de sesión de usuarios.
- Gestión de torneos, partidos y equipos.
- Acciones administrativas (crear, editar y eliminar entidades).
- Manejo de errores y respuestas estructuradas.

En conjunto, esta implementación proporciona una base sólida, segura y modular sobre la que se apoya el funcionamiento del sistema desde el lado del servidor.

2.2.3 IMPLEMENTACIÓN DEL FRONT-END

Para la implementación del frontend se ha utilizado Vue.js, un framework moderno y eficiente que facilita la creación de aplicaciones empresariales en TypeScript. El objetivo del frontend es ofrecer una interfaz intuitiva y fluida que permita a los usuarios tener una buena experiencia de usuario.

Para ello hemos seguido los siguientes puntos:

- Configurar proyecto con Vue 3 e Ionic para construir las vistas.
- Desarrollar componentes/vistas para:
  - Registro e inicio de sesión de usuarios.

- Panel de administración de torneos: creación/edición/eliminación de torneos, equipos y partidos.
- Visualización para alumnos (también visible para administradores): listado de torneos en los que son espectadores y calendario interactivo de partidos.

2.2.4 INTERFAZ DE USUARIO

- Diseñar una UI clara e intuitiva.
- Incluir formularios sencillos para que el administrador pueda crear, editar o borrar, torneos, equipos y pueda gestionar sus partidos.
- Probar la usabilidad y ajustar colores, tipografías y navegación para garantizar una experiencia ágil.

2.2.5 DESPLIEGUE Y DOCUMENTACIÓN

- Despliegue con Docker y AWS
- Documentar endpoints en Swagger UI (Springdoc).
- Grabar un video breve donde se explique el flujo: login, acceso como alumno a vista torneos, equipos y calendario, y rol administrador para gestión de torneos.

2.2.6 MEJORA DE LA COMUNICACIÓN Y SEGUIMIENTO

- Garantizar que la información de partidos (fecha/hora/equipos) se actualice en tiempo real para alumnos.

3. RECURSOS

Los recursos que hemos usado para desarrollar nuestra aplicación web, son los siguientes:

Visual Studio Code es un editor de código fuente gratuito y multiplataforma de Microsoft que ofrece resaltado de sintaxis, depuración integrada, terminal y soporte extensible mediante plugins. Es ligero y configurable, ideal tanto para pequeños scripts como para proyectos más complejos

Vue es un framework progresivo de JavaScript para construir interfaces de usuario interactivas y reactivas. Se centra en la capa de vista y permite crear componentes reutilizables con una sintaxis sencilla.

TypeScript es un lenguaje de programación que extiende a JavaScript añadiendo tipado estático y otras funcionalidades propias de lenguajes más estrictos como Java o C#. Es desarrollado por Microsoft.

Spring Tools es un conjunto de extensiones para IDEs (Eclipse, VS Code) que facilitan el desarrollo con Spring, ofreciendo plantillas, autocompletado y herramientas de depuración específicas. Su objetivo es acelerar la creación y configuración de proyectos Spring Boot.

Spring Boot es un framework de Java que simplifica la creación de aplicaciones independientes al proporcionar configuración automática y convenciones predefinidas. Permite levantar rápidamente servicios REST y microservicios con mínimo código boilerplate.

Java es un lenguaje de programación de propósito general, orientado a objetos y muy usado en desarrollo backend, aplicaciones móviles (Android), sistemas empresariales y más.

MySQL Workbench es una herramienta gráfica oficial para diseño, modelado y administración de bases de datos MySQL, que incluye edición de SQL, diseño de esquemas y gestión de conexiones.

MySQL es un sistema de gestión de bases de datos relacional de código abierto que utiliza SQL para definir, consultar y administrar datos. Es ampliamente utilizado para almacenar y recuperar información en aplicaciones web.

GitHub Desktop es una aplicación gratuita y de código abierto que ofrece una interfaz gráfica para trabajar con repositorios Git, facilitando tareas como clonar, hacer commits, crear ramas y sincronizar con GitHub sin necesidad de utilizar la línea de comandos.

Docker es una plataforma open-source de containerización que construye imágenes ligeras con tu aplicación y sus dependencias y las ejecuta en contenedores aislados, garantizando portabilidad y reproducibilidad en cualquier entorno.

AWS es una plataforma de servicios en la nube bajo demanda que incluye S3 para almacenamiento de objetos escalable y duradero y EC2 para instancias virtuales configurables según tus necesidades.

Figma es una herramienta online para diseñar interfaces y prototipos interactivos. Permite colaborar en tiempo real con otros usuarios desde cualquier lugar.

4. TAREAS

Como parte de nuestras tareas planificadas, diseñamos y desarrollamos todas las vistas esenciales de IESPORTS: la pantalla de inicio de sesión y registro, el panel de administración con gestión de torneos, equipos y partidos, la sección para alumnos espectadores y el calendario interactivo.

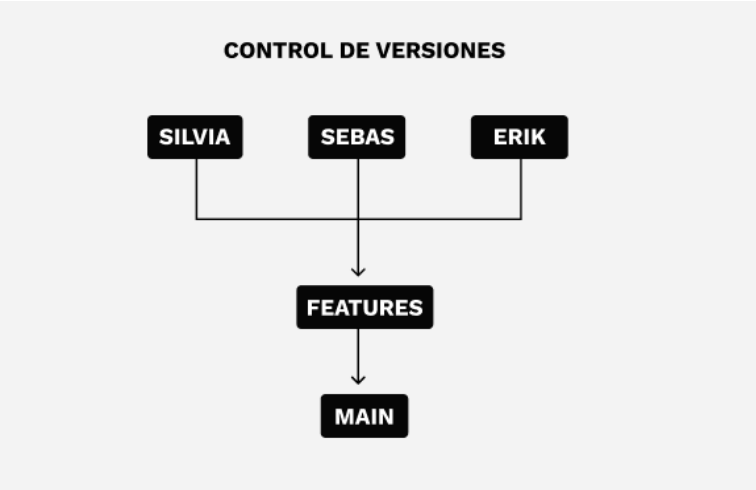
Para cumplir estos objetivos, dedicamos tiempo a investigar y formarnos en las tecnologías clave (Vue 3, Ionic, Spring Boot, Axios), y optimizamos las consultas a la base de datos aplicando filtros directamente en Vue, reduciendo la carga en el servidor.

En la fase final, reestructuramos el código para extraer composables reutilizables, mejoramos la accesibilidad y el diseño visual, e integramos Swagger con Springdoc para generar la documentación de la API.

Distribuimos las tareas entre los miembros del equipo según nuestras fortalezas:

- **Back-end y API REST**: definición de entidades, servicios y controladores en Spring Boot.
- **Front-end y UI**: desarrollo de componentes, layouts e integración con Ionic.
- **Pruebas y despliegue**: validación de flujos, pruebas de usabilidad y configuración del entorno de producción.

Para coordinar el trabajo, seguimos este flujo de Git para control de versiones:



- Repositorio único con dos módulos (backend y frontend).
- Cada desarrollador trabaja en su propia rama, nombrada con su nombre.
- Al completar funcionalidades, abrimos pull requests a la rama de integración **features,** donde realizamos tests y revisiones.
- Tras validar en **features**, fusionamos en **main** solo las versiones estables, listas para despliegue.

Para la documentación, asignamos a cada miembro secciones concretas según su especialidad, asegurando una cobertura detallada y coherente de todo el proyecto.

5. DIAGRAMA TEMPORAL

Para realizar un buen trabajo en equipo creamos una planificación semanal para el inicio del TFG.

La primera semana, planificamos las tareas y realizamos reuniones para definir objetivos y proponer ideas.

La segunda semana, definimos los diagramas de uso, clases, relacionar y E/R y comenzamos la programación básica de la aplicación web.

La tercera semana, nos dedicamos a desarrollar las primeras vistas de la interfaz, estableciendo la navegación y el diseño inicial de las pantallas clave.

La cuarta semana, continuamos con el desarrollo de funcionalidades y realizamos pruebas para verificar su correcto funcionamiento; cada avance estable lo subíamos a la rama main de Git.

En abril centramos nuestros esfuerzos en el desarrollo de los endpoints necesarios y en la creación de las vistas necesarias para ellos.

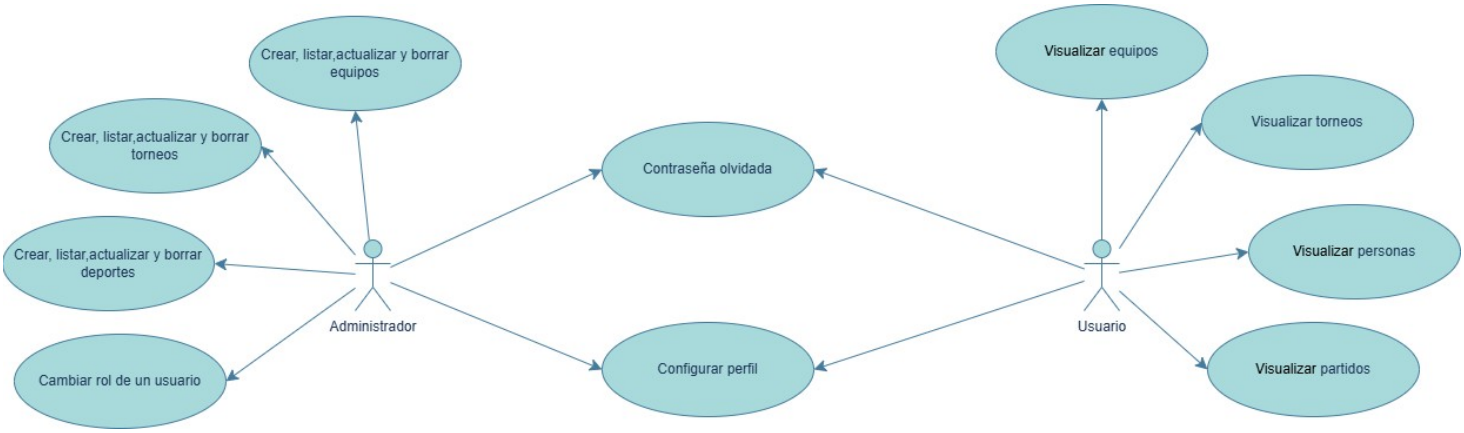
En mayo acabamos con la integración del backend con el frontend y corregimos algunos errores detectados, optimizando la comunicación entre ambas capas.

En junio desplegamos la aplicación web, redactamos la documentación, mejoramos los diagramas y planificamos futuras ampliaciones para nuevas funcionalidades.

6. CAPÍTULOS DE DESARROLLO

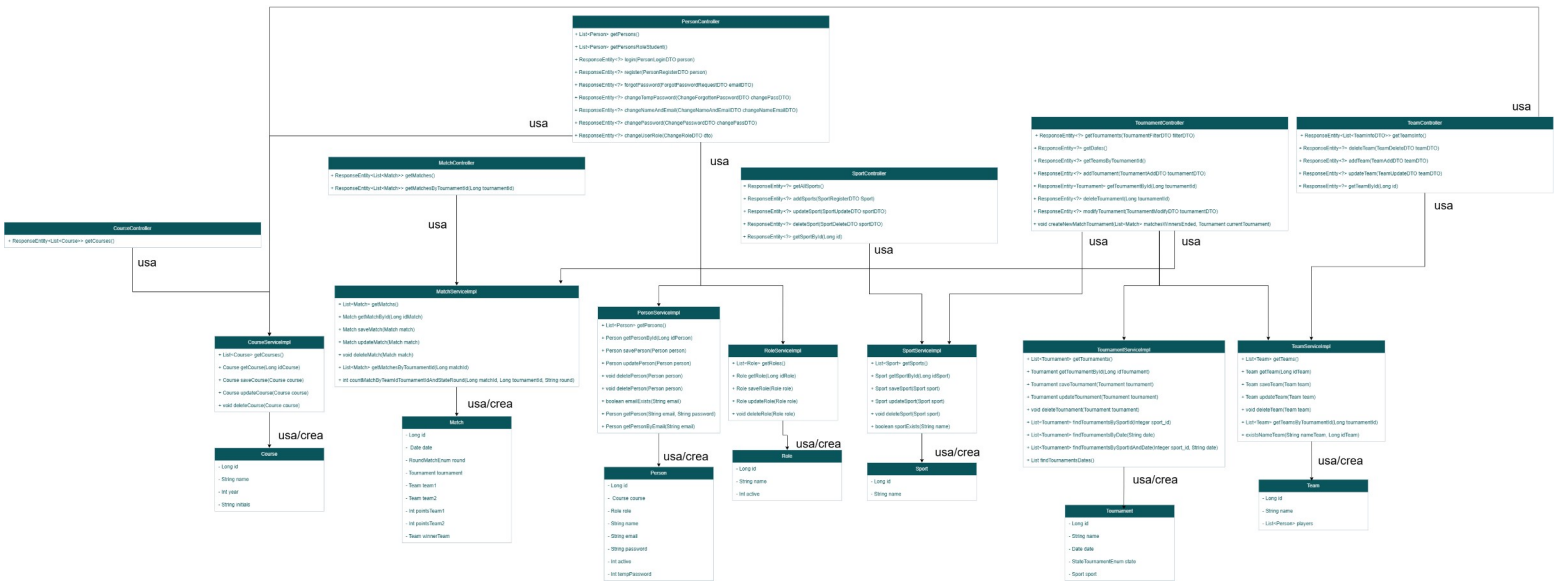
6.1 DIAGRAMA DE CASOS DE USO

Para el desarrollo de nuestra aplicación, es esencial identificar los casos de uso principales de los distintos tipos de usuarios. Esto nos ayuda a entender y satisfacer sus necesidades de forma eficiente. A continuación, presentamos el diagrama elaborado para esta finalidad, que muestra las interacciones entre los usuarios y el sistema, resaltando las funcionalidades clave a implementar.



6.2 DIAGRAMA DE CLASES

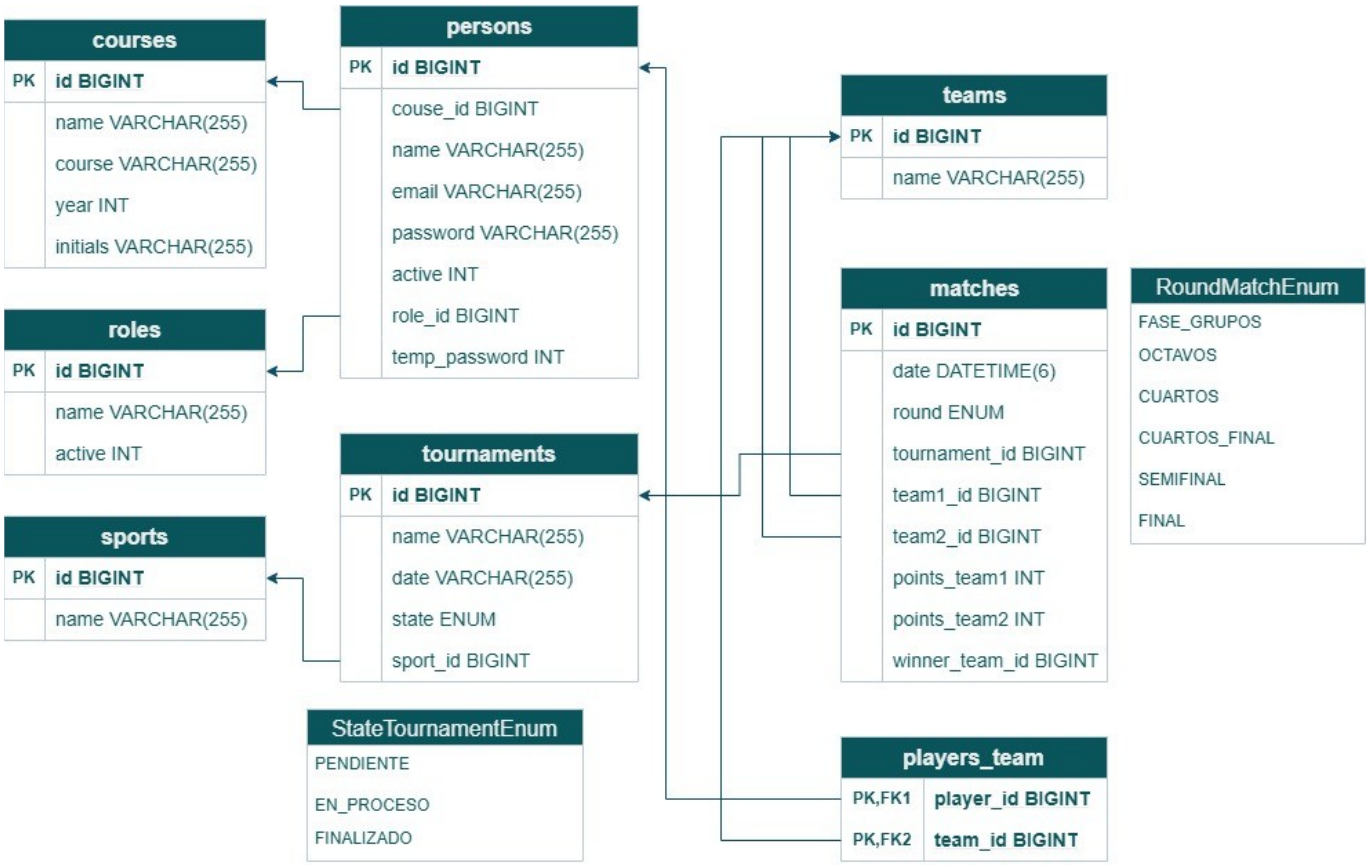
Para estructurar correctamente nuestro sistema, es vital definir las clases principales que lo compondrán. Esto garantiza un código organizado y una interacción efectiva entre sus elementos. A continuación, presentamos el diagrama de clases diseñado, donde se representan las relaciones, atributos y métodos de cada clase.



6.3 DIAGRAMA RELACIONAL

Hemos diseñado un modelo entidad-relación representa las entidades principales y sus vínculos de forma clara y ordenada.

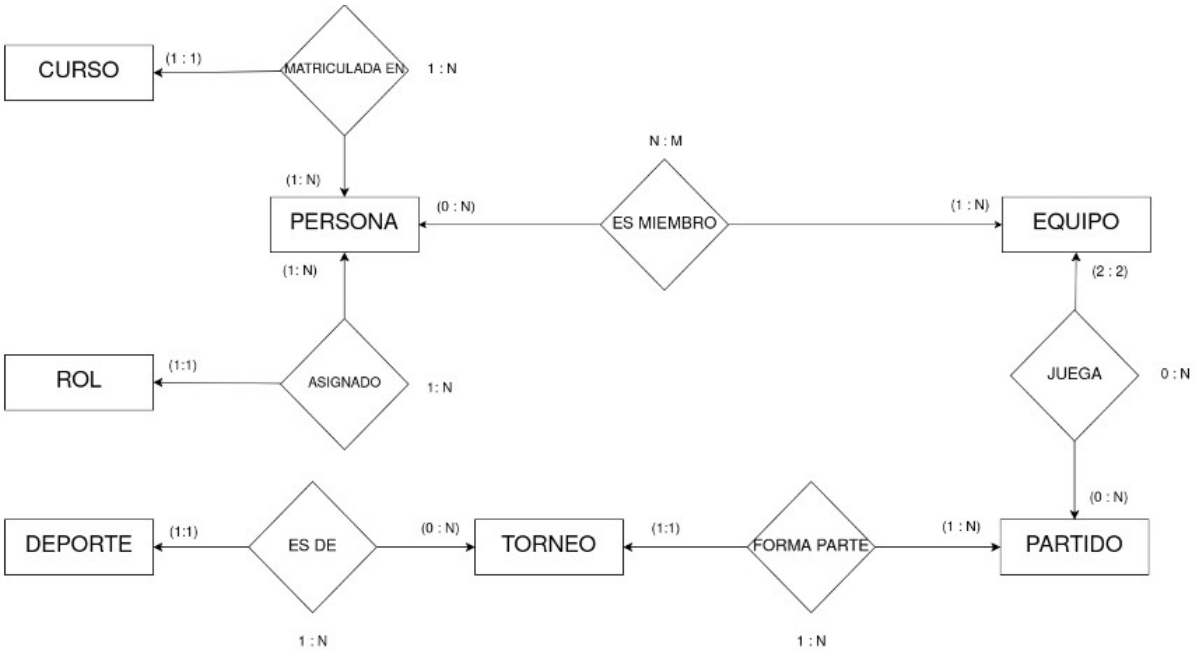
Este modelo facilita la comprensión de las asociaciones entre los datos, permitiendo una implementación eficiente en la base de datos, además, promueve la integridad referencial y simplifica futuras ampliaciones del esquema.



6.4 DIAGRAMA E/R

Hemos diseñado un modelo de datos que captura con precisión las entidades y sus interacciones, garantizando una gestión clara y eficiente de la información. Se estructura de la siguiente manera:

- Un usuario está matriculado en un curso con un rol específico y puede, opcionalmente, participar en uno o varios equipos.
- Los equipos pueden contar con uno o varios miembros y pueden haber jugado partidos, aunque no necesariamente.
- Los partidos se juegan entre dos equipos, partidos los cuáles son jugados en un torneo.
- Un torneo corresponde exclusivamente a un deporte y puede incluir partidos ya disputados y por disputar.



6.5 INNOVACIONES

6.5.1 INNOVACIONES FRONTEND

- **Ficheros composables en Vue 3:** lógica reusable extraída en composables para gestionar el efecto de carga global y el comportamiento del scroll de la navbar (adaptación a puntos de ruptura y bloqueo de desplazamiento en vistas específicas).

TS useNavbarVisibility.ts ×

src > composables > TS useNavbarVisibility.ts > ...

```ts
/**
 * Composable que une:
 *  - visibilidad por scroll (ionScroll)
 *  - visibilidad automática en rutas distintas de "/"
 */
export function useNavbarVisibility() {
  const route = useRoute();
  const showNavbarOnScroll = ref(false);

  // Lógica original de scroll:
  function handleScroll(event: CustomEvent) {
    const el = (event.target as any).getScrollElement
      ? event.target
      : (event.target as any).scrollElement;
    el.getScrollElement().then((scrollEl: HTMLElement) => {
      const top = scrollEl.scrollTop;
      const w = window.innerWidth;
      if (w >= 1440) showNavbarOnScroll.value = top > 900;
      else if (w >= 1024) showNavbarOnScroll.value = top > 700;
      else if (w >= 768) showNavbarOnScroll.value = top > 1000;
      else showNavbarOnScroll.value = top > 600;
    });
  }

  // Siempre mostrar en rutas ≠ "/"
  const alwaysShowNavbar = computed(() => route.path !== '/');

  // Flag combinado
  const showNav = computed(() => alwaysShowNavbar.value || showNavbarOnScroll.value);

  onMounted(() => {
    document.addEventListener('ionScroll', handleScroll as EventListener);
  });
  onUnmounted(() => {
    document.removeEventListener('ionScroll', handleScroll as EventListener);
  });

  return { showNav, handleScroll };
}
```

Lógica reutilizable para mostrar navbar con distintos puntos de ruptura.

- **Animaciones con Partículas:** uso de tsparticles para fondos animados y transiciones ligeras que mejoran la experiencia sin degradar el rendimiento.

```
LoginView.vue ×
src > views > LoginView.vue > {} template > ion-page > ion-content > vue-particles#tsparticles
1  <template>
2    <ion-page>
3      <ion-content>
4        <!-- <div id="vanta-login" class="vanta-bg"></div> -->
5        <vue-particles
6          id="tsparticles"
7          @particles-loaded="particlesLoaded"
8          :options="{
9            background: {
10             color: {
11               value: '#255766'
12             }
13           },
14           fpsLimit: 80,
15           interactivity: {
16             events: {
17               onClick: {
18                 enable: true,
19                 mode: 'push'
20               },
21               onHover: {
22                 enable: true,
23                 mode: 'repulse'
24               },
25             },
26             modes: {
27               bubble: {
28                 distance: 400,
29                 duration: 2,
30                 opacity: 0.2,
31                 size: 20
32               },
33               push: {
34                 quantity: 4
35               },
36               repulse: {
37                 distance: 200,
38                 duration: 0.2
39               }
40             }
41           },
42           particles: {
43             color: {
44               value: '#cee6ed'
45             },
```

Configuración, con el diseño. Colores y duraciones para el fondo animado

- **Estructura de componentes y layouts:** separación clara entre layouts (esqueleto de páginas) y ui (componentes de interfaz genéricos), facilitando la escalabilidad y el mantenimiento.

```
<!-- Lista dinámica de torneos filtrados -->
  <TournamentCard
    v-for="(matchesList, tournamentId) in groupedMatches"
    :key="tournamentId"
    :matches="matchesList"
  />
```

Ejemplo de uso de un componente al que se le pasa una lista partidos generando una tarjeta por cada torneo

- **Services con Axios en el front-end:** cada entidad de la base de datos cuenta con su propio módulo service que agrupa las llamadas HTTP (GET, POST, PUT, DELETE), manteniendo los componentes Vue limpios y centrados en la presentación.

```
function addTournament(TournamentAddDTO: TournamentAddDTO) {

  const url = `${import.meta.env.VITE_URL_API}/tournament/addTournament`;

  return new Promise((resolve, reject) => {
    axios.post(url, TournamentAddDTO)
      .then(response => {
        resolve(response.data);
      })
      .catch(error => {
        console.error('Error al añadir un nuevo torneo:', error);
        reject(error);
      });
  });
}
```

Ejemplo de una llamada a la API con axios

- **Bloqueos de ruta en Vue Router:** configuración de router.beforeEach para comprobar en cada navegación los metadatos requiresAuth y requiresAdmin, recuperando el usuario de localStorage y redirigiendo a Login si no está autenticado, al Home si carece de permisos suficientes, o al panel de administrador (HomeAdmin) cuando su rol sea “Administrador”, garantizando así un control de acceso basado en perfil.

```
TS router.ts ×
src > TS router.ts > routes > meta
3   const routes = [
36    {
37      path: '/homeAdmin',
38      name: 'HomeAdmin',
39      component: () => import('./views/HomeAdmin.vue'),
40      meta: {
41        requiresAuth: true,          // Debe estar logueado
42        requiresAdmin: true          // Y ser administrador
43      }
44    },
45  ]
46
47  const router = createRouter({
48    history: createWebHistory(),
49    routes
50  })
51
52  // 1. Añade este bloque justo después de crear el router:
53  router.beforeEach((to, _from, next) => {
54    // 2. Recuperamos el usuario del localStorage
55    const userJson = localStorage.getItem('usuario')
56    const user = userJson ? JSON.parse(userJson) : null
57
58    // 3. Si la ruta exige autenticación y no hay usuario, vamos a Login
59    if (to.meta.requiresAuth && !user) {
60      return next({ name: 'Login' })
61    }
62
63    // 4. Si la ruta exige admin y el usuario no es “Administrador”, vamos al Home normal
64    if (to.meta.requiresAdmin && (!user || user.role.name !== 'Administrador')) {
65      return next({ name: 'Home' })
66    }
67
68    // 5. Si es admin y va a Home, lo mandamos a HomeAdmin ***
69    if (user && user.role.name === 'Administrador' && to.name === 'Home') {
70      return next({ name: 'HomeAdmin' })
71    }
72
73    // 6. Si pasa todas las comprobaciones, continúa
74    return next()
75  })
76
77  export default router
78
```

Ejemplo de bloqueo de rutas en el fichero router.ts

- **Uso de componentes Ionic:** integración de la librería Ionic en Vue para aprovechar componentes nativos (IonCard, IonList, IonButton, IonModal, etc.), garantizando una interfaz coherente, responsiva y optimizada para dispositivos móviles.

```
<!-- POP UP HAS OLVIDADO CONTRASEÑA START -->
<ion-modal class="forgot-password-modal" :is-open="showForgotPassword" @did-dismiss="showForgotPassword = false" :backdrop-dismiss="false">
  <ion-header class="modal-header">
    <ion-toolbar>
      <ion-title>Recuperar contraseña</ion-title>
      <ion-buttons slot="end">
        <ion-button fill="clear" @click="showForgotPassword = false">
          <span class="material-symbols-outlined">close</span>
        </ion-button>
      </ion-buttons>
    </ion-toolbar>
  </ion-header>

  <ion-content class="ion-padding">
    <span v-if="errores.emailForgotPassword" class="error-msg">{{ errores.emailForgotPassword }}</span>
    <ion-input
      v-model="forgotEmail"
      type="email"
      placeholder="Introduce tu correo"
      fill="outline"
      class="custom-input"
      :class="{ 'error-border': errores.emailForgotPassword }"
    />

    <ion-button class="enviar" expand="block" @click="sendEmailRecovery">
      Enviar correo de recuperación
    </ion-button>

    <ion-button class="clean" expand="block" @click="forgotEmail = ''; errores = {}">
      Limpiar
    </ion-button>
  </ion-content>
</ion-modal>
```

Uso de componentes de Ionic

```
import { IonInput, IonButton, IonIcon, IonContent, IonPage, IonSelect, IonSelectOption } from '@ionic/vue';
```

Componentes de Ionic importados correctamente

6.5.2 INNOVACIONES BACKEND

- **Documentación automática con Springdoc OpenAPI:** generación de Swagger UI a partir de anotaciones en Spring Boot para facilitar pruebas y mantenimiento de la API REST.

Se configura la dependencia en el pom.

Este es el ejemplo de la clase curso con la documentación añadida.

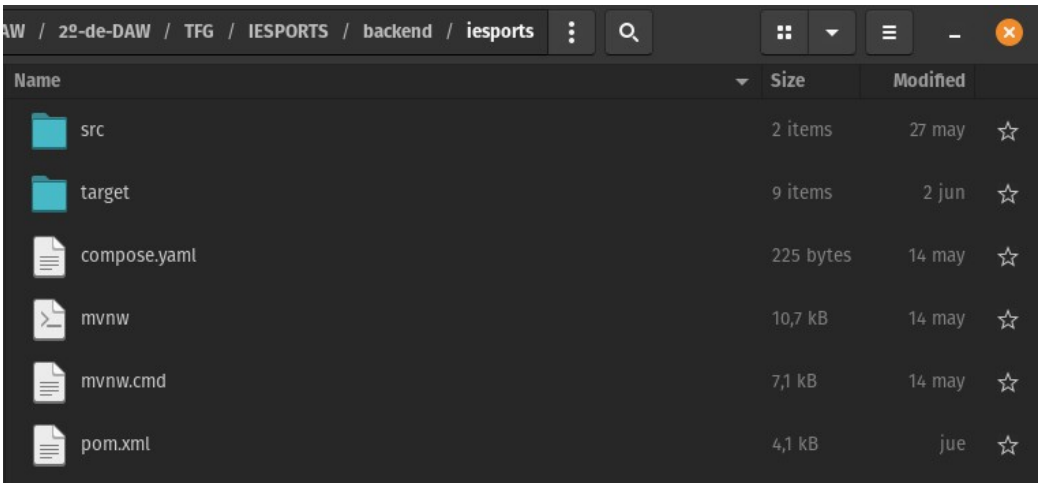
```java
package com.iesports.controller;

import java.util.List;

@RestController
@RequestMapping("/course")
@Tag(name = "Gestión de cursos", description = "Endpoints para la gestión de cursos")
public class CourseController {

    @Autowired
    private CourseServiceImpl cs;

    @Operation(summary = "Obtener la lista de todos los cursos")
    @ApiResponses({
        @ApiResponse(
            responseCode = "200",
            description = "Lista de cursos obtenida correctamente",
            content = @Content(
                mediaType = "application/json",
                array = @ArraySchema(schema = @Schema(implementation = Course.class)),
                examples = @io.swagger.v3.oas.annotations.media.ExampleObject(
                    value = "[{\"id\":1,\"name\":\"Matemáticas\"}, {\"id\":2,\"name\":\"Historia\"}]"
                )
            )
        ),
        @ApiResponse(
            responseCode = "204",
            description = "No hay cursos disponibles",
            content = @Content
        )
    })
    @GetMapping("/getCourses")
    public ResponseEntity<List<Course>> getCourses() {

        List<Course> courses = cs.getCourses();

        if (courses.isEmpty()) {
            return ResponseEntity.status(HttpStatus.NO_CONTENT).body(courses);
        }

        return ResponseEntity.status(HttpStatus.OK).body(courses);
    }

}
```

```xml
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.7.0</version>
</dependency>
```

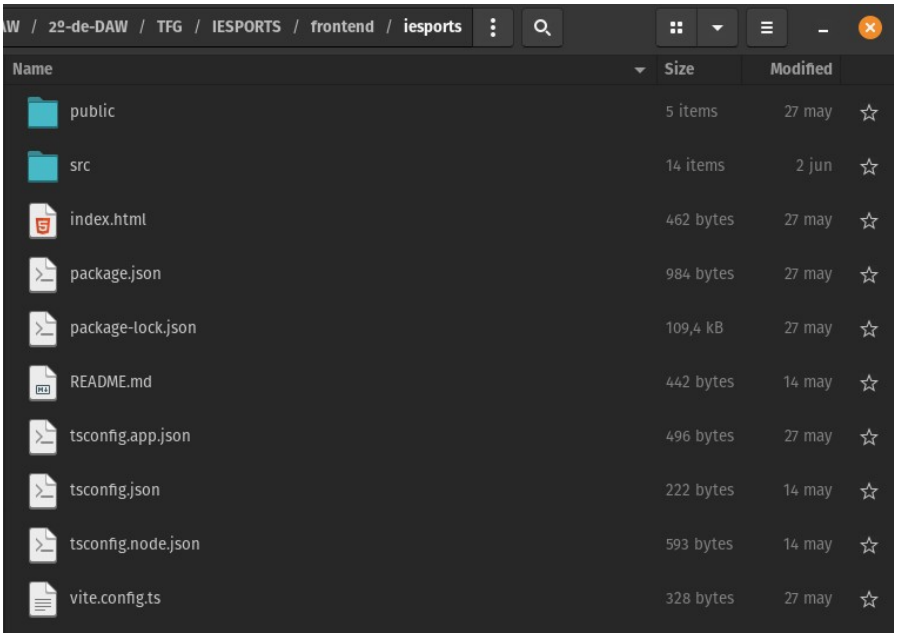
- **Arquitectura front-back modulada**: back-end en Spring Boot y SPA en Vue/Ionic comunicados por API REST, permitiendo despliegues y escalado independientes.

Estructura general de las carpetas.

Estructura del back-end.



Estructura del front-end.



- Manejo de errores en controlador: consiste en una construcción de una lista tipo Map<String, String> con los errores y retorno de un **ResponseEntity**.
  La estructura consiste en un cuerpo **JSON** de pares clave (campo afectado) → valor (definición del error), informando al cliente de forma clara y estructurada sobre los fallos de validación.

  Se crea el Map.

```
Map<String, String> errors = new HashMap<>();
```

  Se van validando los errores y en caso de fallar se añaden a la lista Map.

```
if (ps.emailExists(person.getEmail())) {
    System.err.println("El email " + person.getEmail() + " ya existe");
    errors.put("email", "El email " + person.getEmail() + " ya existe");
}

if (!person.getPassword1().equals(person.getPassword2())) {
    System.err.println("Las contraseñas no coinciden");
    errors.put("password2", "Las contraseñas no coinciden");
}

if (errors.size() > 0) {
    return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(errors);
}
```

- Uso de DTOs para crear objetos con solo los datos que utilice el cliente, evitando exponer todo el contenido de la clase interna.

```
package com.iesports.dto;

import jakarta.validation.constraints.Email;

public class ChangeNameAndEmailDTO {

    @NotNull(message = "La persona no puede ser nula")
    private Long personId;

    @NotBlank(message = "El nombre es obligatorio")
    private String name;

    @Email(message = "El email no es válido")
    @NotBlank(message = "El email es obligatorio")
    private String email;

    public Long getPersonId() {
        return personId;
    }

    public void setPersonId(Long personId) {
        this.personId = personId;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    @Override
    public String toString() {
        return "ChangeNameAndEmailDTO [personId=" + personId + ", name=" + name + ", email=" + email + "]";
    }
```

Ejemplo de un dto de la aplicación.

- Para el envió de correos se utiliza una dependencia en el **pom**:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-mail</artifactId>
</dependency>
```

Esta dependencia se configura en el **application.properties**. En este caso pertenece a gmail y se ha tenido que configurar una contraseña para poder ser utilizado por la aplicación y que no de salte excepción..

```
#configuracion del el servicio de correo
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=noreply.iesports@gmail.com
spring.mail.password=por-motivos-de-seguridad-no-se-muestra-la-contraseña
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
```

Por último se configura una clase para poder utilizarlo en el programa, en la carpeta services nombrada MailService.

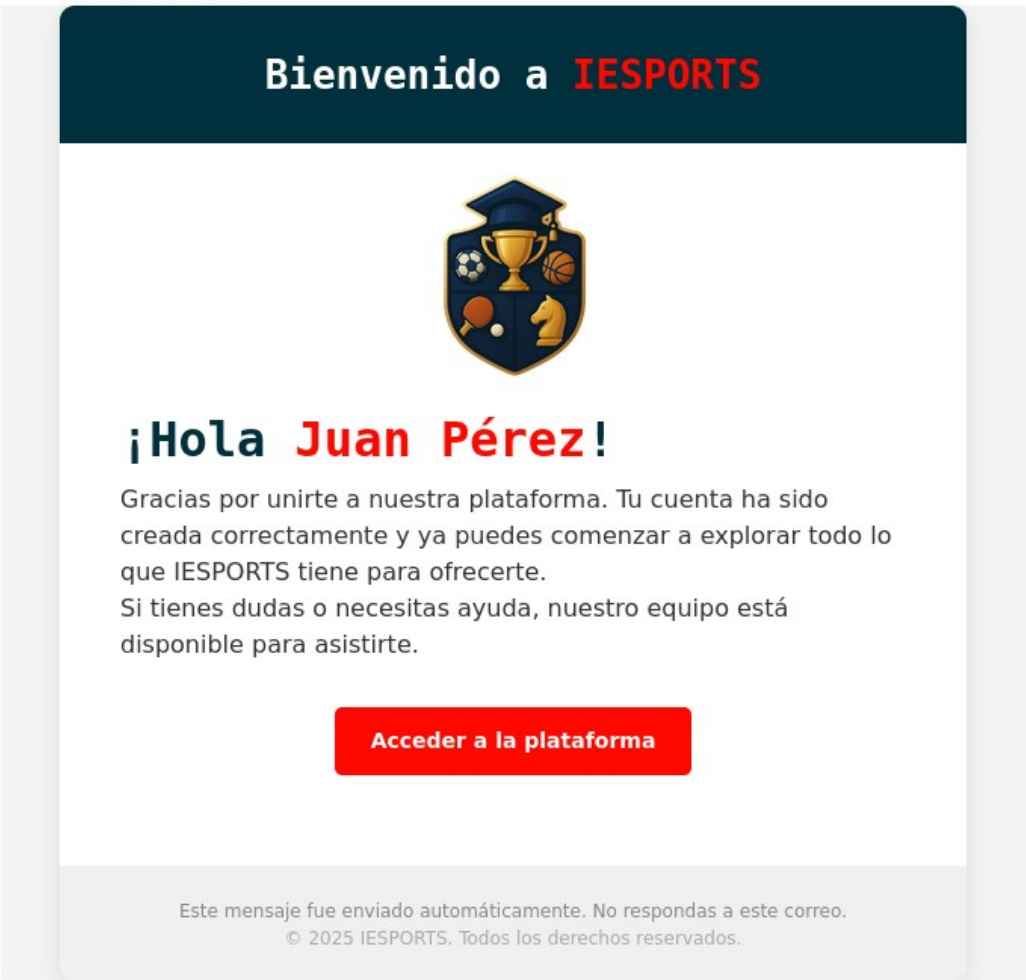
```
package com.iesports.dao.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.core.io.FileSystemResource;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.mail.javamail.MimeMessageHelper;
import org.springframework.stereotype.Service;

import jakarta.mail.MessagingException;
import jakarta.mail.internet.MimeMessage;
import java.io.File;

@Service
public class MailService {

    @Autowired
    private JavaMailSender mailSender;
```

- Envío de correos electrónicos automáticos a los usuarios con mensajes de bienvenida personalizados.

```
public void sendMailForgotPassword(String toEmail, String code, String userName) {
    MimeMessage mimeMessage = mailSender.createMimeMessage();

    try {
        MimeMessageHelper helper = new MimeMessageHelper(mimeMessage, true, "UTF-8");
        helper.setTo(toEmail);
        helper.setSubject("Contraseña temporal IESPORTS");

        String htmlContent = "<html>\n" +
                "  <body style=\"font-family: Arial, sans-serif; background-color: #f5f6fa; padding: 20px;\">\n" +
                "    <div style=\"max-width: 600px; margin: auto; background-color: #ffffff; padding: 30px; border-radius: 12px; box-shadow: 0 4px 10px rgba(0, 0, 0, 0.1);\">\n" +
                "      <!-- Cabecera -->\n" +
                "      <div style=\"background-color: #002f3d; padding: 25px; border-radius: 10px 10px 0 0; text-align: center;\">\n" +
                "        <h1 style=\"color: #ffffff; margin: 0;\">Recuperación de Contraseña</h1>\n" +
                "      </div>\n" +
                "      <!-- Cuerpo del mensaje -->\n" +
                "      <div style=\"padding: 20px;\">\n" +
                "        <p style=\"font-size: 16px; color: #333;\">Hola <strong>"+userName+"</strong>,</p>\n" +
                "        <p style=\"font-size: 16px; color: #333;\">\n" +
                "          Hemos generado una <b>contraseña temporal</b> para ti:\n" +
                "        </p>\n" +
                "        <div style=\"background-color: #fcebea; padding: 15px; border-left: 5px solid #ff0800; margin: 20px 0; border-radius: 5px;\">\n" +
                "          <p style=\"font-size: 18px; color: #ff0800; font-weight: bold; margin: 0;\">Tu nueva contraseña temporal es: <span>"+code+"</span></p>\n" +
                "        </div>\n" +
                "        <p style=\"font-size: 16px; color: #333;\">\n" +
                "          Te recomendamos cambiar esta contraseña lo antes posible por una de tu preferencia. Podrás hacerlo mediante la solicitud de cambio de contraseña que te saltará iniciando sesión con esta contraseña temporal.\n" +
                "        </p>  \n" +
                "      </div>\n" +
                "      <!-- Pie -->\n" +
                "      <div style=\"text-align: center; font-size: 12px; color: #aaa;  border-top: 1px solid #eee;\">\n" +
                "        <p style=\"font-size: 14px; color: #777; margin-bottom: 20px;\">\n" +
                "          Este mensaje ha sido enviado automáticamente. Por favor, no respondas a este correo.\n" +
                "        </p>\n" +
                "      </div>\n" +
                "      <div style=\"text-align: center; font-size: 12px; color: #ffffff; padding: 20px; border-top: 1px solid #eee; background-color: #002f3d; border-radius:0 0 10px 10px;\">\n" +
                "       IESPORTS - Tu instituto, tu torneo, tu victoria\n" +
                "      </div>\n" +
                "    </div>\n" +
                "  </body>\n" +
                "</html>";


        helper.setText(htmlContent, true);
        mailSender.send(mimeMessage);
    } catch (MessagingException e) {
        e.printStackTrace();
    }
}
```

Subject: ¡Bienvenido a nuestra plataforma!

Bienvenido a IESPORTS



- Notificaciones por correo para recuperación de contraseña, facilitando la experiencia y seguridad del usuario.

```java
public void sendWelcomeEmail(String toEmail, String userName) {
    MimeMessage mimeMessage = mailSender.createMimeMessage();

    try {
        MimeMessageHelper helper = new MimeMessageHelper(mimeMessage, true, "UTF-8");
        helper.setTo(toEmail);
        helper.setSubject("¡Bienvenido a nuestra plataforma!");

        // Your image path (adjust if needed)
        FileSystemResource image = new FileSystemResource(
            new File("src/main/resources/static/images/logo.png")
        );

        String htmlContent = "<!DOCTYPE html>\n" +
            "<html lang=\"es\">\n" +
            "<head>\n" +
            "    <meta charset=\"UTF-8\">\n" +
            "    <title>Bienvenido a IESPORTS</title>\n" +
            "</head>\n" +
            "<body style=\"margin: 0; padding: 0; background-color: #f4f4f4; font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;\">\n" +
            "    <table width=\"100%\" cellpadding=\"0\" cellspacing=\"0\" style=\"background-color: #f4f4f4; padding: 40px 0;\">\n" +
            "        <tr>\n" +
            "            <td align=\"center\">\n" +
            "                <table width=\"600\" cellpadding=\"0\" cellspacing=\"0\" style=\"background-color: #ffffff; border-radius: 10px; overflow: hidden; box-shadow: 0 4px 16px rgba(0, 0, 0, 0.1);\">\n" +
            "                    <tr>\n" +
            "                        <td style=\"background-color: #002f3d; padding: 30px; text-align: center;\">\n" +
            "                            <h1 style=\"color: #ffffff; margin: 0; font-size: 26px;\">Bienvenido a <span style=\"color: #ff0800;\">IESPORTS</span></h1>\n" +
            "                        </td>\n" +
            "                    </tr>\n" +
            "                    <tr>\n" +
            "                        <td style=\"text-align: center; padding: 20px;\">\n" +
            "                            <img src=\"cid:logoImage\" alt=\"Logo IESPORTS\" style=\"max-width: 100px;\">\n" + // Reference by CID
            "                        </td>\n" +
            "                    </tr>\n" +
            "                    <tr>\n" +
            "                        <td style=\"padding: 0 40px 30px 40px; text-align: left;\">\n" +
            "                            <h2 style=\"color: #002f3d;\">¡Hola <b style=\"color: #ff0800;\">" + userName + "</b>!</h2>\n" +
            "                            <p style=\"font-size: 16px; color: #333;\">Gracias por unirte a nuestra plataforma. Tu cuenta ha sido creada correctamente y ya puedes comenzar a explorar todo lo que IESPORTS tiene para ofrecerte.
            "                            <p style=\"font-size: 16px; color: #333;\">Si tienes dudas o necesitas ayuda, nuestro equipo está disponible para asistirte.</p>\n" +
            "                            <div style=\"text-align: center; margin: 30px 0;\">\n" +
            "                                <a href=\"https://tusitio.com/login\" style=\"background-color: #ff0800; color: #ffffff; padding: 12px 24px; text-decoration: none; font-weight: bold; border-radius: 5px; display: inline-block;\
            "                            </div>\n" +
            "                        </td>\n" +
            "                    </tr>\n" +
            "                    <tr>\n" +
            "                        <td style=\"background-color: #f0f0f0; padding: 20px; text-align: center;\">\n" +
            "                            <p style=\"font-size: 12px; color: #888;\">Este mensaje fue enviado automáticamente. No respondas a este correo.</p>\n" +
            "                            <p style=\"font-size: 12px; color: #aaa;\">&copy; 2025 IESPORTS. Todos los derechos reservados.</p>\n" +
            "                        </td>\n" +
            "                    </tr>\n" +
            "                </table>\n" +
            "            </td>\n" +
            "        </tr>\n" +
            "    </table>\n" +
            "</body>\n" +
            "</html>";
```

Subject: Contraseña temporal IESPORTS

Recuperación de Contraseña

Hola **Juan Pérez**,
Hemos generado una **contraseña temporal** para ti:

Tu nueva contraseña temporal es: cN3gN0wy

Te recomendamos cambiar esta contraseña lo antes posible por una de tu preferencia. Podrás hacerlo mediante la solicitud de cambio de contraseña que te saltará iniciando sesión con esta contraseña temporal.

Este mensaje ha sido enviado automáticamente. Por favor, no respondas a este correo.

IESPORTS - Tu instituto, tu torneo, tu victoria

- Contraseñas cifradas mediante BCrypt para garantizar la seguridad y protección de los datos de los usuarios.

  En la clase SecurityConfig se inicializa el passwordEncoder.

```
@Bean
BCryptPasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

  Con esta función podemos encriptar contraseñas de nuevos usuarios.

```
String passwordEncripted = passwordEncoder.encode(person.getPassword1());
```

  O también podemos comparar la contraseña que llega al endpoint con la almacenada en la base de datos.

```
!passwordEncoder.matches(person.getPassword(), foundPerson.getPassword())
```

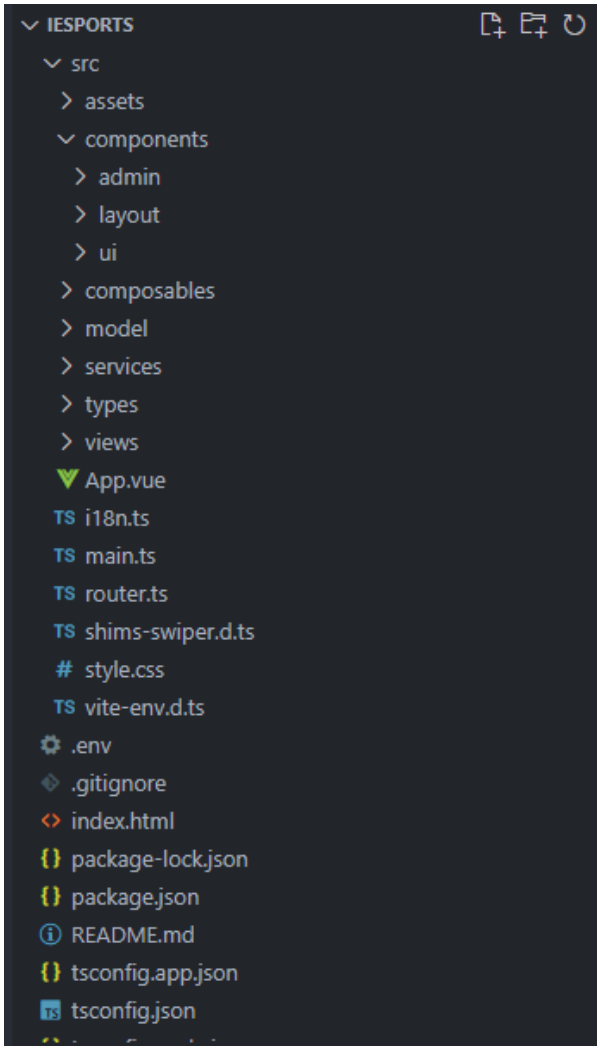
Ejemplo de contraseñas encriptadas en la base de datos

password
$2a$10$GE7GA/T2MOA0DEbDskLih.daAuvRkC3t.7nZbpKTSRKYqvTBEX7Ey
$2a$10$GE7GA/T2MOA0DEbDskLih.daAuvRkC3t.7nZbpKTSRKYqvTBEX7Ey
$2a$10$GE7GA/T2MOA0DEbDskLih.daAuvRkC3t.7nZbpKTSRKYqvTBEX7Ey
$2a$10$GE7GA/T2MOA0DEbDskLih.daAuvRkC3t.7nZbpKTSRKYqvTBEX7Ey
$2a$10$GE7GA/T2MOA0DEbDskLih.daAuvRkC3t.7nZbpKTSRKYqvTBEX7Ey
$2a$10$GE7GA/T2MOA0DEbDskLih.daAuvRkC3t.7nZbpKTSRKYqvTBEX7Ey
$2a$10$GE7GA/T2MOA0DEbDskLih.daAuvRkC3t.7nZbpKTSRKYqvTBEX7Ey
$2a$10$GE7GA/T2MOA0DEbDskLih.daAuvRkC3t.7nZbpKTSRKYqvTBEX7Ey
$2a$10$GE7GA/T2MOA0DEbDskLih.daAuvRkC3t.7nZbpKTSRKYqvTBEX7Ey

6.6 CÓDIGO Y ESTRUCTURA

6.6.1 ESTRUCTURA FRONTEND

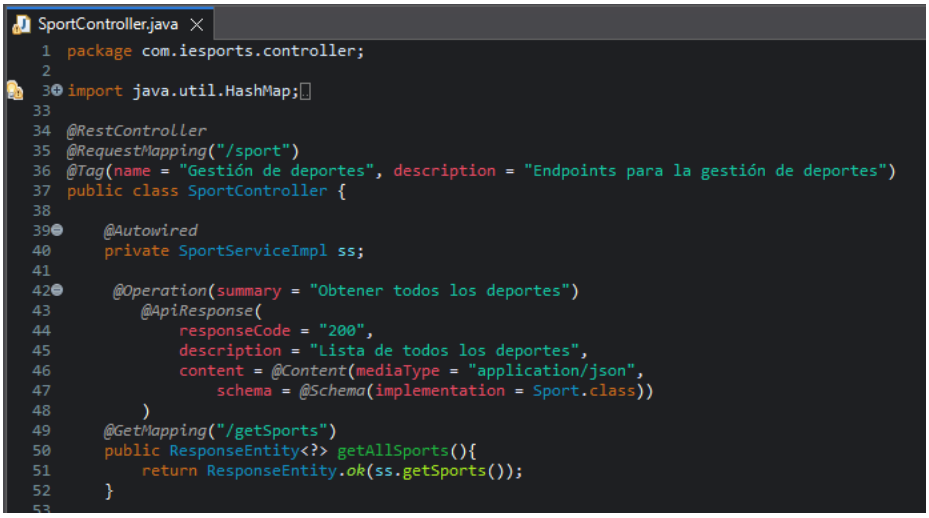
- assets: en esta carpeta se guardan imagenes y ficheros css reutilizables
- components:
  - **layout:** esqueletos de página
  - **ui:** componentes genéricos de interfaz
  - **admin:** componentes específicos del panel de administración
- compostables: lógica reusable (efecto de carga, scroll de navbar, etc.)
- model: clases de dominio y carpeta dto con los DTOs correspondientes
- services: módulos Axios para llamadas a la API, uno por cada entidad
- types: declaración de módulos de Vanta.js y otras extensiones
- views: vistas asociadas a las rutas definidas en router.ts
- Por debajo de estas carpetas se encuentran los archivos de configuración global del proyecto (router.ts, vite.config.ts, tsconfig.ts, .env, main.ts, etc).



6.6.2 ESTRUCTURA BACKEND

El proyecto está estructurado en varias carpetas que organizan el código según cada componente/funcionalidad dentro de la arquitectura de la aplicación. Esta división permite una mejor mantenibilidad y escalabilidad del sistema.

- **config:** Contiene la clase CorsConfig, encargada de configurar los permisos CORS de la aplicación. Gracias a esta clase, se permite la comunicación entre el backend que consta de una aplicación en Spring Boot y el frontend que consta de una aplicación Vue como por ejemplo alojado en localhost:5173 o 5174 (para el despliegue en local). Esta configuración podrá variar dependiendo del despliegue.
- **controller:** Agrupa todos los controladores de la aplicación. Estas clases se encargan de recibir las peticiones HTTP, aplicar directamente la lógica de negocio, y utilizar los servicios para acceder a los datos persistentes.

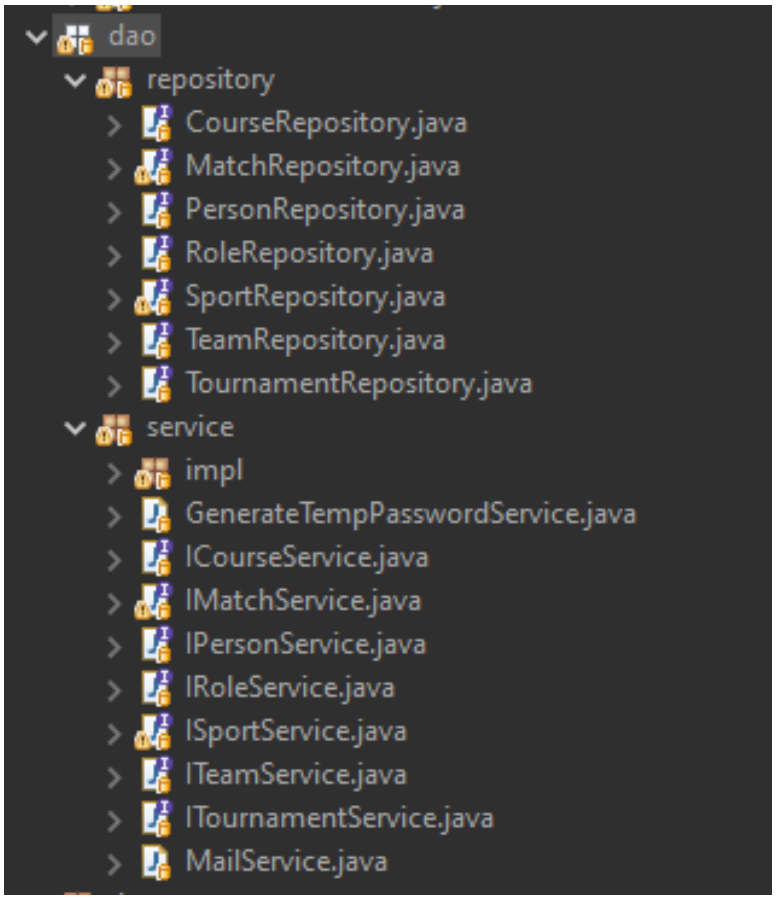


```
SportController.java
1  package com.iesports.controller;
2
3  import java.util.HashMap;
33
34 @RestController
35 @RequestMapping("/sport")
36 @Tag(name = "Gestión de deportes", description = "Endpoints para la gestión de deportes")
37 public class SportController {
38
39     @Autowired
40     private SportServiceImpl ss;
41
42      @Operation(summary = "Obtener todos los deportes")
43         @ApiResponse(
44             responseCode = "200",
45             description = "Lista de todos los deportes",
46             content = @Content(mediaType = "application/json",
47                 schema = @Schema(implementation = Sport.class))
48         )
49     @GetMapping("/getSports")
50     public ResponseEntity<?> getAllSports(){
51         return ResponseEntity.ok(ss.getSports());
52     }
```

Ejemplo de un controller de la aplicación.

- Eje

- **dao:** Esta carpeta contiene a su vez dos subcarpetas:



Ejemplo estructura dao de la aplicación.

- **repository:** Contiene todos los repositorios que extienden de JpaRepository, proporcionando acceso directo a la base de datos mediante métodos predefinidos y consultas personalizadas.

```
TournamentRepository.java
1  package com.iesports.dao.repository;
2
3  import java.util.List;
9
10 public interface TournamentRepository extends JpaRepository<Tournament, Long>{
11
12     @Query(value = "SELECT * FROM tournaments WHERE sport_id = ?1 sport_id  ORDER by date DESC", nativeQuery = true)
13     List<Tournament> findTournamentsBySportId(Integer sport_id);
14
15     @Query(value = "SELECT * FROM tournaments WHERE date = ?1 date  ORDER by date DESC", nativeQuery = true)
16     List<Tournament> findTournamentsByDate(String date);
17
18     @Query(value = "SELECT * FROM tournaments WHERE sport_id = ?1 sport_id  and date =?2 date  ORDER by date DESC", nativeQuery = true)
19     List<Tournament> findTournamentsBySportIdAndDate(Integer sport_id, String date);
20
21     @Query(value = "SELECT DISTINCT date FROM tournaments ORDER by date DESC", nativeQuery = true)
22     List<String> getTournamentsDates();
23
24 }
```

Ejemplo código TournamentRepository

- **service:** Aquí se definen las interfaces de servicio (por ejemplo, TeamServiceImpl, etc.) y sus respectivas implementaciones. Estas clases no contienen lógica de negocio, sino que se encargan únicamente de interactuar con los repositorios para acceder a la base de datos, funcionando como intermediarios técnicos entre los controladores y los repositorios.

ITournamentService.java

```java
package com.iesports.dao.service;

import java.util.List;

public interface ITournamentService {

    public List<Tournament> getTournaments();

    public Tournament getTournamentById(Long idTournament);

    public Tournament saveTournament(Tournament tournament);

    public Tournament updateTournament(Tournament tournament);

    public void deleteTournament(Tournament tournament);

    public List<Tournament> findTournamentsBySportId(Integer sport_id);

    public List<Tournament> findTournamentsByDate(String date);

    public List<Tournament> findTournamentsBySportIdAndDate(Integer sport_id, String date);

    public List<Tournament> findTournamentsDates();

}
```

Ejemplo código ITournamentService

- **dto:** Contiene los DTOs (Data Transfer Objects) utilizados para transferir datos de forma segura y eficiente entre el backend y el frontend.

MatchDTO.java

```java
package com.iesports.dto;

import java.util.Date;

public class MatchDTO {

    @NotBlank(message = "El id del equipo 1 es obligatorio")
    private Long team1Id;
    @NotBlank(message = "El id del equipo 2 es obligatorio")
    private Long team2Id;

    private Date matchDate;

    public Long getTeam1Id() {
        return team1Id;
    }
    public void setTeam1Id(Long team1Id) {
        this.team1Id = team1Id;
    }
    public Long getTeam2Id() {
        return team2Id;
    }
    public void setTeam2Id(Long team2Id) {
        this.team2Id = team2Id;
    }
    public Date getMatchDate() {
        return matchDate;
    }
    public void setMatchDate(Date matchDate) {
        this.matchDate = matchDate;
    }
}
```

Ejemplo código de un dto (MachDTO)

- **enums:** Esta carpeta contiene las enumeraciones utilizadas en el sistema para representar valores constantes de forma estructurada. En concreto, se incluyen:
  - **RoundMatchEnum:** Define las distintas fases eliminatorias que puede tener un torneo, tales como Octavos, Cuartos de Final, Semifinal y Final. Esto permite organizar y controlar el progreso de los enfrentamientos a lo largo del campeonato.

```java
RoundMatchEnum.java
1 package com.iesports.enums;
2
3 public enum RoundMatchEnum {
4     OCTAVOS,CUARTOS_FINAL,SEMIFINAL,FINAL
5 }
6
```

  - **StateTournamentEnum:** Representa los diferentes estados posibles de un torneo, como Pendiente, Proceso o Finalizado. Esta enumeración facilita el seguimiento del progreso de los torneos y asegura coherencia en el estado de los mismos.

```java
StateTournamentEnum.java
1 package com.iesports.enums;
2
3 public enum StateTournamentEnum {
4     PENDIENTE,PROCESO,FINALIZADO
5 }
```

- **exception:** Alberga la clase **ValidationExceptionHandler**, una clase con la anotación **@ControllerAdvice** que captura las excepciones de validación de formularios y devuelve un mapa de errores en formato clave-valor. Esta clase mejora la gestión de errores en la API, proporcionando mensajes claros al cliente.

```java
ValidationExceptionHandler.java
1  package com.iesports.exception;
2
3  import java.util.HashMap;
10
11 // Esta clase sirve para que los errores de validacion con Anotaciones devuelva un clave valor del error que se indica
12 @ControllerAdvice
13 public class ValidationExceptionHandler {
14
15     @ExceptionHandler(MethodArgumentNotValidException.class)
16     public ResponseEntity<Map<String, String>> handleValidationErrors(MethodArgumentNotValidException ex) {
17         Map<String, String> errors = new HashMap<>();
18
19         ex.getBindingResult().getFieldErrors().forEach(error ->
20             errors.put(error.getField(), error.getDefaultMessage())
21         );
22
23         errors.forEach((campo, mensaje) -> System.err.println("- " + campo + ": " + mensaje));
24
25         return ResponseEntity.badRequest().body(errors);
26     }
27 }
```

- **model:** Aquí se encuentran todas las clases modelo que representan las entidades de la base de datos. Estas clases están anotadas con @Entity y reflejan directamente las tablas de la base de datos.

```java
Tournament.java
package com.iesports.model;
import com.iesports.enums.StateTournamentEnum;

@Entity
@Table(name = "tournaments")
@Schema(description = "Entidad que representa un torneo")
public class Tournament {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Schema(description = "ID único del torneo", example = "1", accessMode = Schema.AccessMode.READ_ONLY)
    private Long id;

    @NotBlank(message = "El nombre del torneo es obligatorio")
    @Schema(description = "Nombre del torneo", example = "Ajedrez Winter")
    private String name;

    @NotNull(message = "La fecha del torneo es obligatoria")
    @Schema(description = "Fecha del torneo en formato YYYY/YY", example = "2022/23")
    private String date;

    @NotNull(message = "El estado del torneo es obligatorio")
    @Enumerated(EnumType.STRING)
    @Schema(description = "Estado actual del torneo", example = "FINALIZADO")
    private StateTournamentEnum state;

    @ManyToOne
    @JoinColumn(name = "sport_id", referencedColumnName = "id")
    private Sport sport;

    public Tournament() {

    }

    public Tournament(Long id,String name, String date, StateTournamentEnum state, Sport sport) {
        super();
        this.id = id;
        this.name = name;
        this.date = date;
        this.state = state;
        this.sport = sport;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }
```

Ejemplo código de un clase (Tournament)

- **security:** Contiene la configuración de seguridad de la aplicación. La clase incluye un bean de BCryptPasswordEncoder para el cifrado de contraseñas.

```java
SecurityConfig.java
package com.iesports.security;

import org.springframework.context.annotation.Bean;

@Configuration
public class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth.anyRequest().permitAll());
        return http.build();
    }

    @Bean
    BCryptPasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

7. RESULTADOS OBTENIDOS

7.1 PROBLEMAS Y SUS SOLUCIONES

Durante el desarrollo de la aplicación, nos enfrentamos a distintos desafíos que fuimos resolviendo progresivamente. Estos son los distintos problemas a los que les hemos ido poniendo solución:

1. Transmisión de datos estructurada

Problema: Al inicio del proyecto no sabíamos cómo hacer llegar la información únicamente necesaria al cliente. Esto generaba incertidumbre en torno a la validación y recogida de datos.
Solución: La implementación de los DTO (Data Transfer Objects) nos permitió simplificar este proceso. Gracias a ellos, pudimos establecer un formato claro y controlado para los datos que se intercambian entre cliente y servidor, minimizando el riesgo de exposición innecesaria de información y reduciendo la complejidad del código.

2. Seguridad del backend expuesto a cualquier origen

Problema: Inicialmente, el backend estaba completamente expuesto y aceptaba peticiones desde cualquier origen, lo cual representaba un riesgo de seguridad considerable.
Solución: A través de la configuración de CORS en el backend, logramos restringir el acceso a peticiones que provengan únicamente de URLs y puertos específicos, asociados al frontend desarrollado con Vue en modo SPA.

3. Recuperación de contraseñas olvidadas

Problema: No contábamos con una forma automatizada para recuperar contraseñas, lo cual podía implicar que un administrador tuviera que intervenir manualmente cada vez que un usuario olvidaba la suya**.**
Solución: Implementamos un servicio de correo electrónico que permite enviar una contraseña temporal al usuario que haya olvidado la suya. Esto automatiza el proceso y descarga al administrador de esa responsabilidad, mejorando la experiencia del usuario y la eficiencia del sistema.

4. Gestión poco clara de errores de validación

Problema: Al inicio, no teníamos una forma estructurada de manejar y devolver errores al frontend cuando ocurrían problemas de validación (como un email ya existente o contraseñas que no coinciden). Esto dificultaba tanto la depuración como la experiencia del usuario, ya que los errores no se comunicaban de forma clara ni unificada.

Solución: Implementamos un Map<String, String> para recolectar todos los errores detectados durante la validación y devolverlos de forma conjunta al frontend con un **ResponseEntity** y el código de estado **HTTP** adecuado. Esto permite mostrar múltiples errores al usuario de forma clara y coherente.

Este enfoque mejora la claridad del código, facilita la identificación de errores y permite que el frontend pueda mostrar mensajes específicos para cada campo problemático.

8. CONCLUSIONES Y PROPUESTAS

Durante el desarrollo de esta aplicación web orientada al seguimiento de torneos, equipos y partidos, se han cumplido todos los objetivos propuestos. La aplicación permite a los usuarios interactuar con contenidos deportivos, consultar información sobre equipos y torneos, y acceder a funcionalidades clave como la gestión de usuarios, autenticación, y visualización de datos relevantes.

A lo largo del proyecto, se han aplicado buenas prácticas en la arquitectura del backend, separando responsabilidades entre controladores y servicios, y mejorando la seguridad con medidas como la configuración de CORS y el uso de validaciones centralizadas. Además, se ha conseguido una buena integración entre el frontend (SPA en Vue) y el backend, lo que proporciona una experiencia fluida para el usuario.

Conclusiones destacadas:

- ➔ Se ha logrado una aplicación funcional y bien estructurada que cumple con los requerimientos básicos.
- ➔ La organización del código ha mejorado significativamente con el tiempo, aplicando principios de modularidad y claridad.
- ➔ Se ha reforzado la seguridad y la gestión de errores, lo que mejora la robustez del sistema.
- ➔ El envío automatizado de contraseñas por correo ha reducido la carga administrativa y ha mejorado la experiencia del usuario.

8.1 POSIBLES MEJORAS

- **Validación del usuario** mediante un código enviado por correo electrónico para activar la cuenta y garantizar la identidad.
- **Roles de usuario** (por ejemplo: jugador, administrador de equipo, profesor y administrador) con diferentes permisos y vistas personalizadas. El propósito sería reducir la carga de trabajo al rol administrador.

8.2 POSIBLES AMPLIACIONES

- **Notificaciones en tiempo real:** para avisar de nuevos partidos, resultados o cambios en torneos. Cada vez que se ponga fecha a un partido, notificar vía email a cada usuario de está notificación.
- **Sistema de comentarios y reacciones:** para aumentar la interacción entre usuarios en torno a partidos y torneos.
- Crear un formulario de solicitud para unirse a equipos, y estas solicitudes le lleguen peticiones al administrador (un apartado para estas peticiones).

9- BIBLIOGRAFÍA Y REFERENCIAS

IONIC: https://ionicframework.com/docs/components

Componentes prediseñados: https://uiverse.io/

Fuentes: https://www.dafont.com/es/theme.php?cat=710

Imágenes: https://www.pexels.com/es-es/

Licencia: https://creativecommons.org/chooser/

Diseño de vistas: https://www.figma.com/

Fondo animado particles: https://youtu.be/cUzihD4JBQU?si=OEbPY2ovuvANHUX5

Java DTO: https://youtu.be/nMIdoGcE5XE?si=aUxC5BN9G1ePyNct

Enviar correo: https://youtu.be/kLMUS0-PznE?si=-wKd_NzQCk13q9qG

10- ANEXOS

10.1 PROTOTIPO DE IESPORTS

Para su diseño hemos utilizado Figma, una herramienta colaborativa y versátil para la creación de interfaces.

El prototipo facilita la iteración rápida de las pantallas y la validación del flujo de navegación con usuarios.

Además, permite identificar posibles mejoras y optimizar la experiencia antes de la implementación.

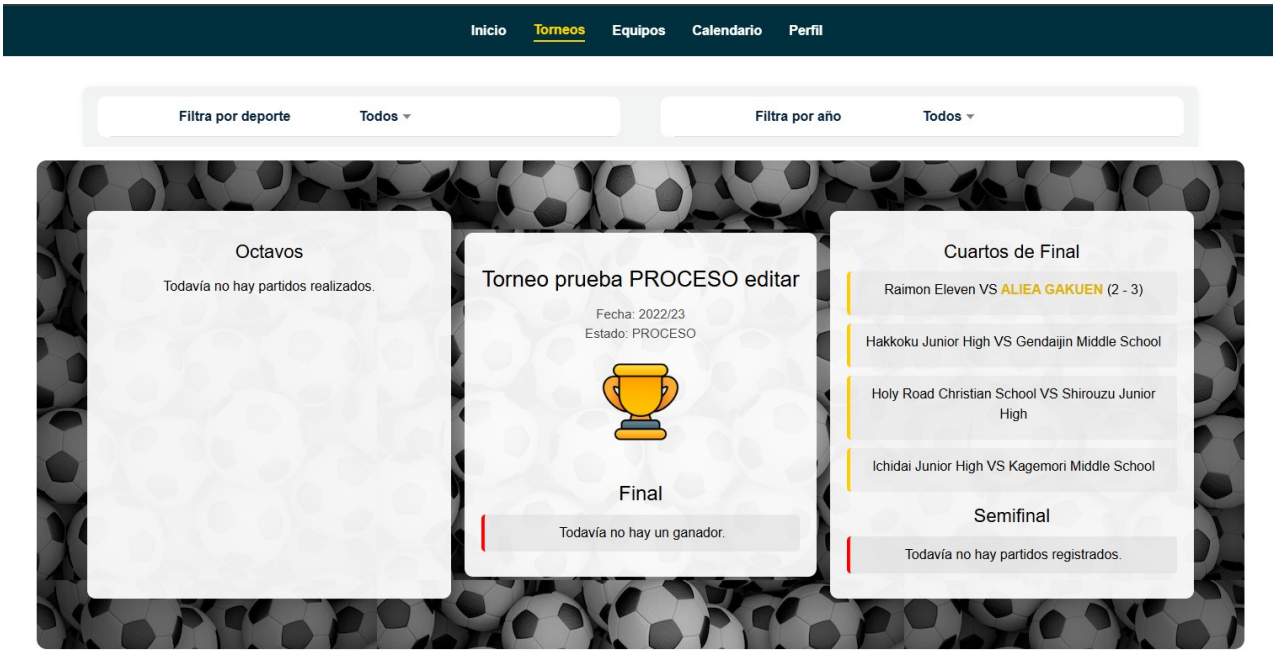
Para acceder al prototipo, haz clic en el siguiente enlace:

https://my.visme.co/view/z413v9y1-color-theory-ebook

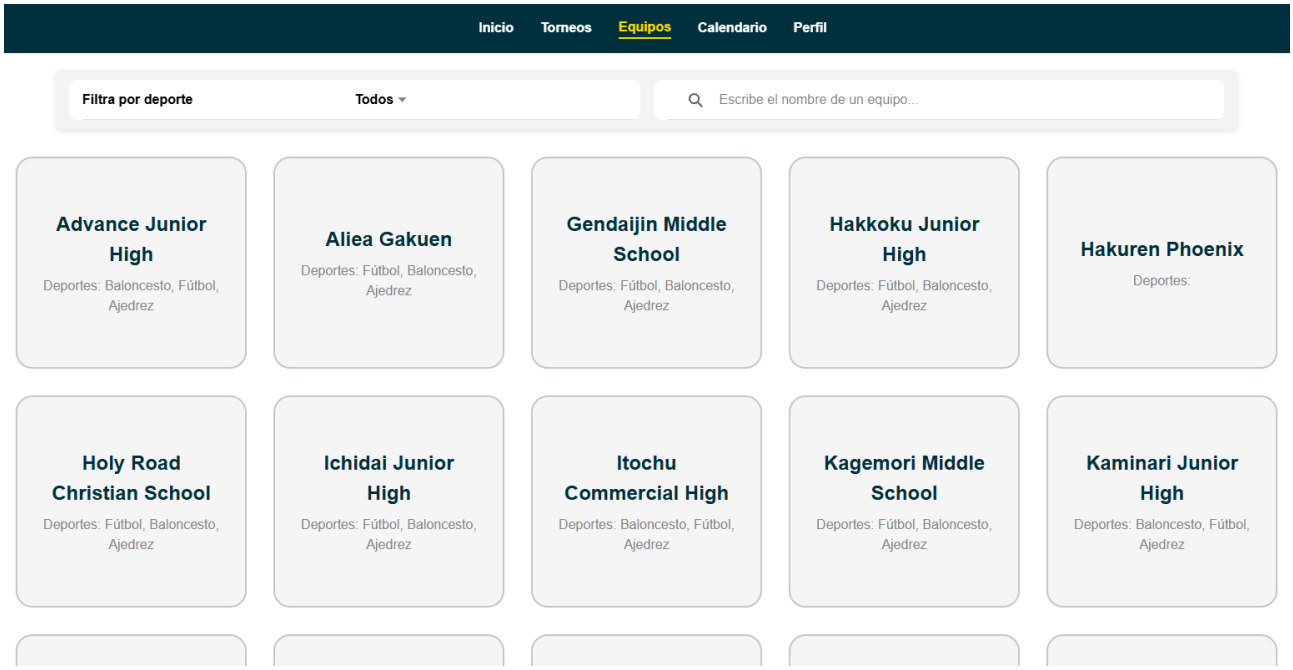
10.2 VISTAS GENERALES DE IESPORTS

En este apartado vamos a mostrar algunas de las vistas de la aplicación web:

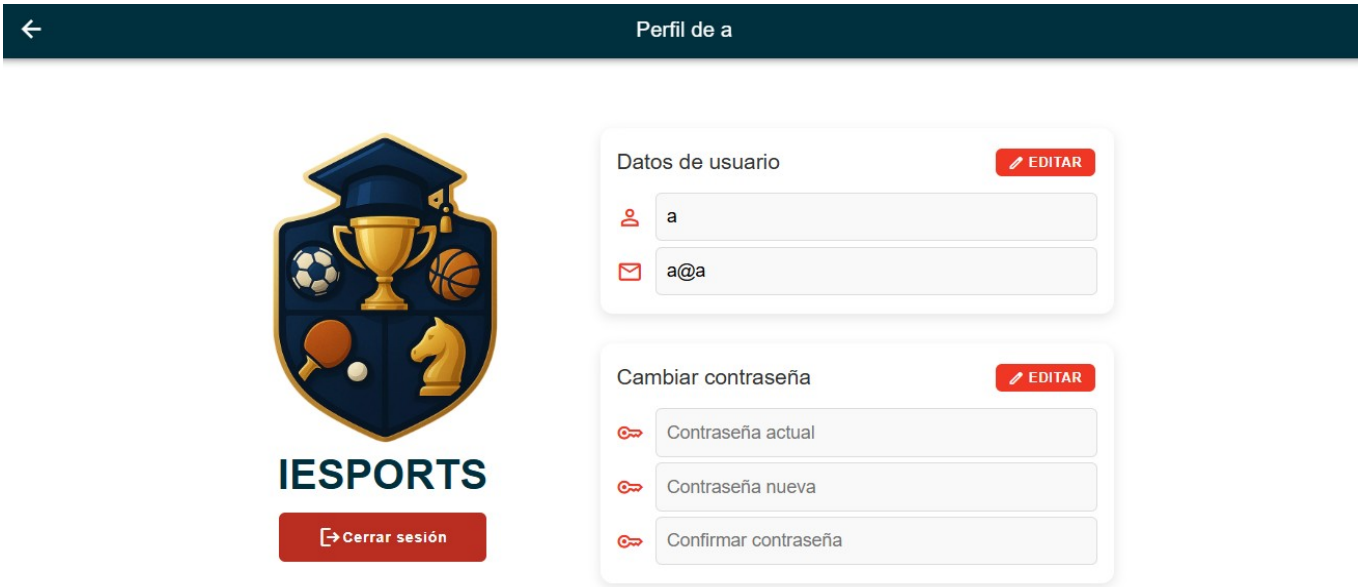
VISTA TORNEOS USUARIO LOGEADO



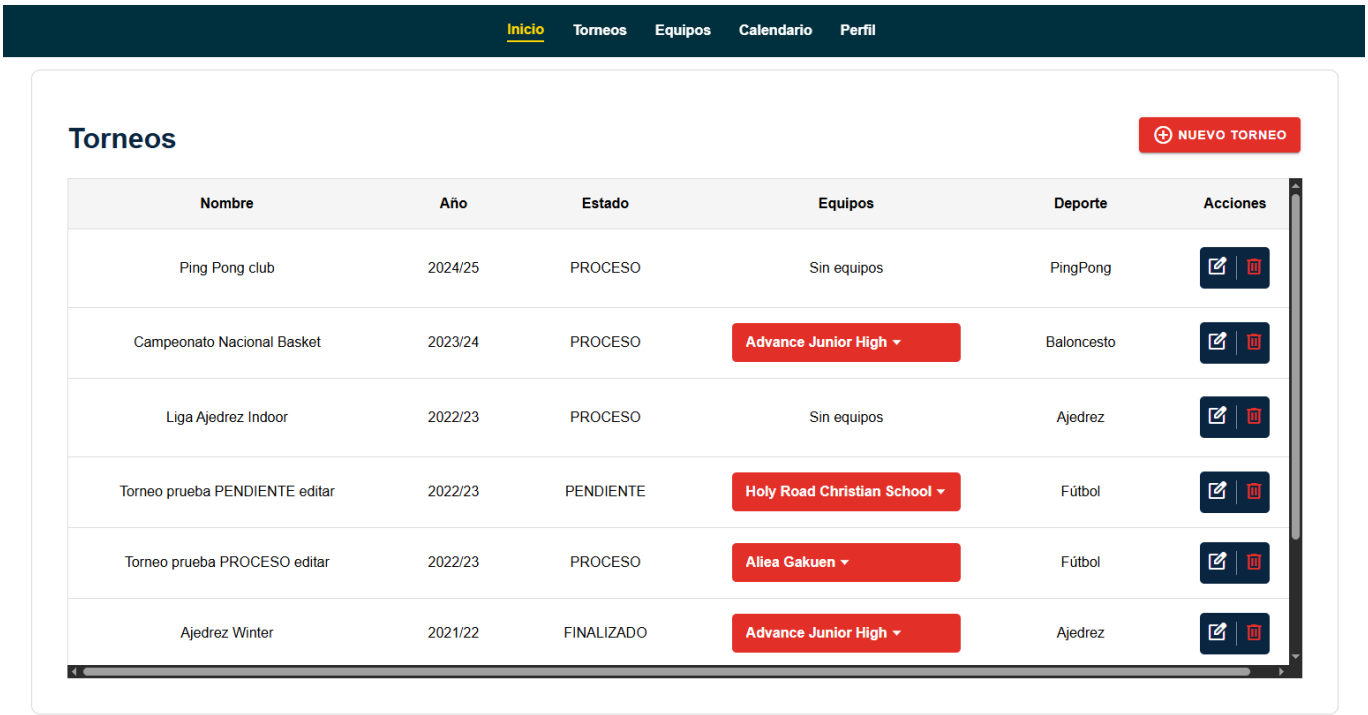
VISTA EQUIPOS USUARIO LOGEADO



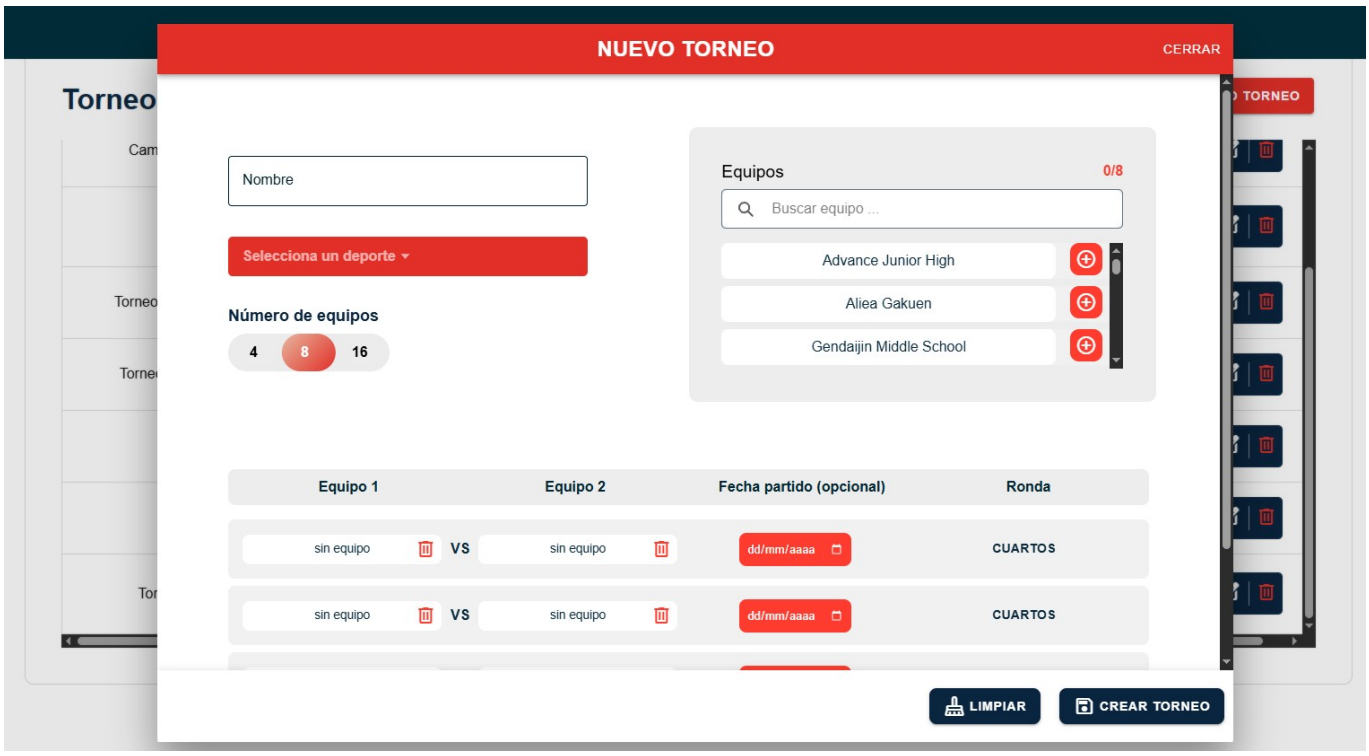
VISTA PERFIL USUARIO LOGEADO



VISTA ADMINISTRADOR PARA GESTIÓN DE TORNEOS



VISTA ADMINISTRADOR PARA GESTIÓN UN NUEVO TORNEO



10.3 LOGO

10.3.1 LOGO

10.3.2 ESLOGAN

IESPORTS

Tu instituto, tu torneo, tu victoria

10.2.3 LOGO CON ESLOGAN